

Models e Views no Django (padrão MVT)

Atividade de pesquisa guiada – Partes 1 e 2

Aluno: Tallys Cesar Gurgel Batista
Curso: Análise e Desenvolvimento de Sistemas
Instituição: IFRN – Campus Pau dos Ferros/RN
Disciplina: Desenvolvimento de Sistemas Web
Professor: Irlan Arley Targino Moreira

Parte 1 – Investigando os Models

1.1. O que é um Model no Django e qual problema ele resolve dentro da arquitetura MVT?

O Model é a camada que descreve a estrutura dos dados da aplicação: cada Model é uma classe Python que representa uma tabela do banco de dados, e cada atributo dessa classe representa uma coluna. Ele resolve o problema de separar “como os dados são organizados e persistidos” do restante da aplicação — a View não precisa saber SQL, e o Template não precisa saber nada sobre banco de dados. Isso mantém a responsabilidade de cada camada bem definida: Model cuida dos dados, View cuida da lógica e regras de negócio, e Template cuida da apresentação.

1.2. O que é ORM e como o Django usa esse conceito para evitar escrever SQL manualmente?

ORM (Mapeamento Objeto-Relacional) é a técnica que traduz classes e objetos do código para tabelas e registros de um banco relacional, e vice-versa. No Django, quando se escreve algo como `Book.objects.all()`, o ORM converte essa chamada internamente em uma instrução SQL equivalente e devolve o resultado já como objetos Python prontos para uso. Isso evita escrever SQL manualmente, deixa o código mais legível e torna a aplicação portátil entre diferentes bancos de dados (SQLite, PostgreSQL, MySQL etc.) sem precisar alterar o código Python.

1.3. Como se declara um Model? Liste pelo menos 5 tipos de campo e explique quando usar cada um.

Um Model é declarado criando uma classe que herda de `models.Model`, onde cada atributo da classe é uma instância de um campo (Field), como no exemplo:

```
class Book(models.Model):  
    title = models.CharField(max_length=200)
```

Alguns tipos de campo e quando usá-los:

Campo	Quando usar
CharField	Textos curtos e de tamanho limitado (nome, título); exige <code>max_length</code> .
TextField	Textos longos sem limite fixo de tamanho (descrição, resumo, biografia).
IntegerField	Números inteiros (ano, quantidade, idade).

BooleanField	Valores verdadeiro/falso (ex.: disponível, ativo).
DateField / DateTimeField	Datas, ou datas com horário (data de publicação, data de cadastro).
ForeignKey	Relacionamento entre tabelas (ex.: um livro pertence a uma editora).

1.4. Para que serve a classe Meta dentro de um Model? Cite pelo menos duas opções configuráveis nela.

A class `Meta`, dentro do `Model`, define configurações de comportamento que não são campos de dados, mas sim metadados sobre a tabela e o próprio model. Duas opções comuns: **ordering**, que define a ordem padrão dos registros ao consultar (ex.: `ordering = ['title']`); e **verbose_name** / **verbose_name_plural**, que define como o model aparece no Django Admin (ex.: “Livro” e “Livros”).

1.5. O que são migrations? Explique a diferença entre makemigrations e migrate.

Migrations são arquivos Python gerados automaticamente que registram, passo a passo, as alterações feitas nos Models (criar tabela, adicionar campo, etc.), funcionando como um histórico versionado do esquema do banco de dados. O comando `makemigrations` **cria** esses arquivos, comparando o estado atual dos Models com o último estado conhecido, mas ainda não altera o banco. Já o comando `migrate` **aplica** essas migrations no banco de dados de fato, executando o SQL correspondente e efetivamente criando ou alterando as tabelas.

Parte 2 – Ligando Models às Views

2.1. Como uma view importa e utiliza um Model para buscar dados do banco?

A view importa o Model a partir do arquivo `models.py` do próprio app, com algo como `from .models import Book`, e usa o manager `objects` para consultar o banco — por exemplo, `Book.objects.all()` — dentro do corpo da função da view, antes de enviar o resultado para o template.

2.2. O que é um QuerySet? Explique a diferença entre Model.objects.all(), .filter() e .get().

`QuerySet` é uma coleção “preguiçosa” (lazy) de objetos que representa uma consulta ao banco: ela só é de fato executada no banco quando é efetivamente usada, como ao ser percorrida em um `for` ou convertida em lista. O método `.all()` retorna um `QuerySet` com todos os registros da tabela. O `.filter(condição=valor)` retorna um `QuerySet` apenas com os registros que atendem à condição informada, podendo ter zero, um ou vários resultados, e continua sendo uma coleção. Já o `.get(condição=valor)` retorna um único objeto específico, não uma lista, e levanta uma exceção `DoesNotExist` caso nada seja encontrado, ou `MultipleObjectsReturned` caso mais de um registro corresponda à condição.

2.3. Como os dados buscados no Model chegam até o Template? Pesquise o dicionário de contexto passado para render().

A view passa um dicionário de contexto como terceiro argumento da função `render(request, 'template.html', contexto)`. Cada chave desse dicionário vira uma variável acessível

diretamente no template por meio da sintaxe `{{ chave }}`. Por exemplo, passando `{'books': books}`, é possível usar `{% for book in books %}` no HTML para percorrer os livros.

2.4. Pesquise o atalho `get_object_or_404`. Por que ele é preferível a um `Model.objects.get()` direto dentro de uma view?

O `get_object_or_404` é preferível porque já trata de forma pronta o cenário em que o objeto buscado não existe: caso o registro não seja encontrado, ele automaticamente levanta um erro `Http404`, exibindo uma página de “não encontrado” amigável ao usuário, em vez de deixar a exceção `DoesNotExist` propagar e gerar um erro 500 (erro interno do servidor). Isso deixa o código da view mais limpo e evita ter que escrever manualmente um bloco `try/except` toda vez que um objeto é buscado diretamente por um identificador.